МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

МЕТОДИЧЕСКИЕ УКАЗАНИЯ

к выполнению лабораторной работы № 3

«ДИНАМИЧЕСКОЕ ПОВЕДЕНИЕ ВЕБ-СТРАНИЦЫ»

ДИСЦИПЛИНА: «ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННЫХ СИСТЕМ»

Преподаватель: Мирошниченко Н.И.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Установка окружения разработки 4
1.1 Расширение Live Server для VS Code 4
1.2 Инструменты разработчика браузера (DevTools) 4
2 Теоретическая часть 6
2.1 DOM и выбор элементов 6
2.2 Изменение содержимого и атрибутов 7
2.3 Стили через JS 8
2.4 События 9
2.5 Всплытие и делегирование событий 10
2.6 Создание и удаление элементов 11
2.7 Работа с формами 12
2.8 Таймеры (setTimeout, setInterval) 13
2.9 Fetch API 13
2.10 JSON 14
3 Порядок выполнения лабораторной работы 16
4 Самостоятельная работа 17
5 Итоговое задание 19
ЗАКЛЮЧЕНИЕ 21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 22

ВВЕДЕНИЕ

Настоящие методические указания предназначены для выполнения лабораторной работы № 3 и охватывают программирование поведения веб-страницы на языке JavaScript: работу с деревом DOM, обработку событий, динамическое создание и удаление элементов, работу с формами, таймеры, а также обмен данными с сервером через Fetch API и формат JSON.

Целью лабораторной работы является освоение базовых приёмов клиентского JavaScript, необходимых для превращения статичной HTML/CSS-вёрстки (полученной в лабораторной работе № 2) в интерактивную страницу: чтение и изменение содержимого элементов, реакция на действия пользователя, динамическое обновление интерфейса и обмен данными с внешним сервером.

Методические указания включают: порядок установки необходимого программного обеспечения, теоретическую часть по каждой из десяти тем работы с листингами примеров, порядок выполнения лабораторной работы, требования к самостоятельной работе и итоговому заданию. Ссылка на задания: https://disk.yandex.ru/d/trP0tdta3healQ.

1 Установка окружения разработки

1.1 Расширение Live Server для VS Code

Live Server — расширение VS Code, поднимающее локальный сервер для статичных файлов и автоматически перезагружающее страницу при изменении файлов проекта. Для лабораторной работы № 3 расширение необходимо по той же причине, что и в лабораторной работе № 2, и дополнительно — потому что тема «Fetch API» требует загрузки страницы именно по протоколу http://, а не через прямое открытие файла (file://): часть браузеров блокирует сетевые запросы со страниц, открытых с диска напрямую.

Порядок установки:

— открыть VS Code;

— перейти в раздел Extensions (иконка квадратов на боковой панели);

— найти расширение «Live Server» (автор Ritwick Dey) через поиск и установить его;

— открыть файл варианта задания (var01.html и т. п.) правым кликом → «Open with Live Server»;

— страница откроется в браузере по адресу вида http://127.0.0.1:5500/....

Статус: обязательно к установке.

1.2 Инструменты разработчика браузера (DevTools)

DevTools — встроенная в браузер панель инструментов разработчика (открывается клавишей F12 или Ctrl+Shift+I). Для лабораторной работы № 3 DevTools становится основным инструментом отладки: вкладка Console используется для вывода значений через console.log() и просмотра ошибок JavaScript, вкладка Elements — для проверки того, что скрипт действительно изменил DOM, а вкладка Network — для просмотра запросов, отправленных функцией fetch() (тема 2.9), включая код ответа сервера и содержимое JSON.

Практический совет: любая ошибка в JavaScript-коде (опечатка в имени переменной, обращение к несуществующему элементу и т. п.) прерывает

выполнение скрипта и печатается красным текстом в Console вместе с номером строки файла — начинать отладку любого TODO следует именно с просмотра этой вкладки.

Статус: обязательно к использованию на протяжении всей лабораторной работы.

2 Теоретическая часть

2.1 DOM и выбор элементов

DOM (Document Object Model) — представление HTML-документа в виде дерева объектов, доступное из JavaScript через глобальный объект document. Каждый тег страницы становится объектом-узлом (Node), с которым можно взаимодействовать: читать и менять содержимое, атрибуты, стили, добавлять и удалять узлы.

Основные способы получить элемент со страницы:

```
// по id — самый быстрый и однозначный способ, id уникален на странице
const title = document.getElementById("main-title");

// по CSS-селектору — вернёт ПЕРВЫЙ подходящий элемент
const card = document.querySelector(".card");

// по CSS-селектору — вернёт ВСЕ подходящие элементы (NodeList)
const items = document.querySelectorAll(".item");
```

querySelector/querySelectorAll принимают любой CSS-селектор — по классу (.card), по id (#main-title), по тегу (li), а также составные селекторы (ul li.active). Это делает их более гибкими, чем getElementById, который работает только по id.

Важное отличие: document.querySelectorAll возвращает NodeList, а не обычный массив. У NodeList есть метод forEach (можно перебирать в цикле как массив), но нет методов map, filter, reduce напрямую — при необходимости NodeList оборачивают в массив через Array.from(list) или [...list]. document.getElementsByClassName и getElementsByTagName (устаревший способ) возвращают ещё один тип — HTMLCollection, у которого нет даже forEach и который, в отличие от NodeList из querySelectorAll, «живой»: автоматически обновляется при изменении DOM.

```
const items = document.querySelectorAll(".item");
items.forEach(function (item) {
  console.log(item.textContent);
});
```

Если элемент с указанным id или селектором отсутствует на странице, getElementById и querySelector возвращают null, а не ошибку. Обращение к свойству null (например, null.textContent) приводит к ошибке во время выполнения — поэтому перед работой с результатом запроса рекомендуется проверять его на null, особенно если элемент существует не всегда (например, появляется динамически):

```
const el = document.getElementById("maybe-missing");
if (el) {
  el.textContent = "Найден";
}
```

→ Переходите к практике: Задание № 1 (папка «Задание № 1»).

2.2 Изменение содержимого и атрибутов

После получения элемента с ним можно работать: менять текст, HTML-содержимое, атрибуты и CSS-классы.

textContent и innerHTML — оба свойства меняют содержимое элемента, но по-разному. textContent воспринимает переданное значение как обычный текст — даже если в строке встречаются символы < и >, они не превратятся в теги, а будут показаны как есть. innerHTML, наоборот, разбирает переданную строку как HTML-разметку и создаёт из неё настоящие элементы:

```
el.textContent = "<b>жирный</b>";  // покажет текст "<b>жирный</b>" буквально
el.innerHTML  = "<b>жирный</b>";  // создаст тег <b> — текст станет жирным
```

Использовать innerHTML с данными, введёнными пользователем (например, из поля формы), небезопасно — внутри такой строки может оказаться вредоносный код (атака XSS). Для вставки простого текста всегда следует предпочитать textContent; innerHTML оправдан только тогда, когда нужно вставить именно готовую HTML-разметку, которую контролирует разработчик.

Атрибуты элемента читаются и меняются методами getAttribute/setAttribute:

```
const link = document.querySelector("a");
link.getAttribute("href");                  // прочитать значение
link.setAttribute("href", "https://ya.ru"); // задать новое значение
```

Для CSS-классов существует отдельный, более удобный интерфейс — classList, с методами add (добавить класс), remove (убрать класс), toggle (переключить — добавить, если класса нет, и убрать, если есть) и contains (проверить наличие класса, вернёт true/false):

```
el.classList.add("active");
el.classList.remove("hidden");
el.classList.toggle("open");
if (el.classList.contains("active")) { /* ... */ }
```

→ Переходите к практике: Задание № 2 (папка «Задание № 2»).

2.3 Стили через JS

Инлайновые CSS-стили элемента доступны через свойство element.style. Имена CSS-свойств с дефисом (background-color) записываются в camelCase (backgroundColor), поскольку дефис в JavaScript — оператор вычитания:

```
const box = document.querySelector("#js-box");
box.style.backgroundColor = "orange";
box.style.width = "200px";   // числовые значения обязательно со
единицей измерения
```

element.style позволяет только ЧИТАТЬ и ПИСАТЬ инлайновые стили — те, что заданы непосредственно через атрибут style у элемента. Стили, заданные через классы или теги в CSS-файле, через element.style не видны. Чтобы прочитать итоговое, уже применённое браузером значение любого CSS-свойства (независимо от того, откуда оно взялось — из файла, из тега <style> или инлайново), используется метод getComputedStyle:

```
const computed = getComputedStyle(box);
console.log(computed.backgroundColor); // фактический цвет, каким его
видит браузер
```

Изменение через element.style работает по тем же правилам специфичности CSS, что и обычные стили из файла: инлайновый стиль имеет очень высокий приоритет, но если правило в CSS-файле задано через ID-

селектор (#toast { display: inline-block; }), оно может перекрыть переключение класса через classList.toggle("hidden"), если у .hidden указано display: none, а у #toast — свой display прямо на том же элементе. ID-селектор (#id) специфичнее класс-селектора (.class), поэтому при конфликте побеждает правило по id — даже если класс .hidden добавлен позже в HTML или в CSS-файле ниже по тексту. Практический вывод: если элемент должен полностью скрываться/показываться через переключение класса, не задавайте конфликтующее свойство display этому же элементу отдельным ID-селектором.

→ Переходите к практике: Задание № 3 (папка «Задание № 3»).

2.4 События

Событие (event) — сигнал о том, что на странице что-то произошло: клик мышью, нажатие клавиши, отправка формы, ввод текста и т. д. Чтобы отреагировать на событие, элементу назначается обработчик — функция, вызываемая браузером в момент события. Современный способ назначить обработчик — метод addEventListener:

```
const btn = document.querySelector("#like-btn");
btn.addEventListener("click", function (event) {
  console.log("Кнопку нажали");
});
```

Частая ошибка новичков — передать в addEventListener не ссылку на функцию, а РЕЗУЛЬТАТ её вызова (с круглыми скобками). btn.addEventListener("click", doSomething()) вызовет doSomething один раз немедленно, при чтении этой строки, а не при клике; правильно — без скобок: addEventListener("click", doSomething).

Браузер передаёт в обработчик объект event с информацией о произошедшем: event.target — элемент, на котором событие реально случилось (подробнее в 2.5), event.preventDefault() — метод, отменяющий стандартное поведение браузера для этого события (например, отправку формы или переход по ссылке).

Разные типы событий срабатывают в разные моменты, и их легко перепутать. Ниже — прямое сравнение двух наиболее часто путаемых пар событий, применительно к работе с формами (тема 2.7):

— "input" — срабатывает на самом текстовом поле (<input>) при КАЖДОМ изменении его содержимого, то есть практически на каждый введённый или удалённый символ; удобно для проверки в реальном времени (например, счётчика оставшихся символов);

— "submit" — срабатывает на элементе <form> ОДИН РАЗ, в момент отправки формы (клик по кнопке type="submit" или Enter в поле); именно на этом событии обычно выполняют финальную проверку всех полей и вызывают event.preventDefault(), чтобы страница не перезагружалась;

— "click" — срабатывает на конкретном элементе (кнопке, ссылке, любом другом) при клике именно по нему; в отличие от "submit", не привязано к форме и не связано с отправкой данных;

— "change" — срабатывает на select, checkbox и radio при изменении выбранного значения, но, в отличие от "input", не при каждом символе, а при потере элементом фокуса после изменения (для чекбоксов и radio — сразу при клике).

→ Переходите к практике: Задание № 4 (папка «Задание № 4»).

2.5 Всплытие и делегирование событий

Всплытие (bubbling) — механизм, при котором событие, произошедшее на вложенном элементе, «поднимается» вверх по дереву DOM и повторно срабатывает на каждом его родителе, вплоть до document. Если обработчик клика повешен и на дочернем <li>, и на родительском <ul>, то клик по <li> вызовет сначала обработчик на <li>, а затем — обработчик на <ul>.

Внутри обработчика важно различать два свойства события: event.target — элемент, на котором событие произошло НА САМОМ ДЕЛЕ (например, конкретный <li>, по которому кликнули), и event.currentTarget — элемент, на котором СЕЙЧАС выполняется данный обработчик (то есть тот, на который

сам addEventListener был повешен, например <ul>). Внутри обработчика event.currentTarget всегда равен this.

На всплытии основан приём делегирования событий: вместо того чтобы вешать по обработчику на каждый из множества похожих дочерних элементов (что расточительно и не работает для элементов, добавленных позже), вешают ОДИН обработчик на общего родителя, а внутри него определяют, по какому именно дочернему элементу кликнули, через event.target и метод closest():

```
const list = document.querySelector("#todo-list");
list.addEventListener("click", function (event) {
  const li = event.target.closest("li");
  if (!li) return;              // клик мимо элементов списка
  li.classList.toggle("done");
});
```

closest("li") ищет ближайший элемент, начиная с event.target и поднимаясь вверх, который соответствует указанному селектору — это надёжнее, чем полагаться на то, что event.target — это точно <li>: если внутри <li> лежит, например, , а кликнули именно по нему, event.target будет ссылаться на , а closest("li") всё равно вернёт нужный <li>.

Делегирование особенно ценно при динамическом создании элементов (тема 2.6): обработчик, повешенный на родителя один раз при загрузке страницы, продолжает работать и для элементов, которые появятся внутри него позже — в отличие от прямого обработчика, назначенного конкретному элементу, который для новых элементов пришлось бы вешать заново.

Если всплытие события нежелательно (например, клик по кнопке внутри карточки не должен вызывать обработчик клика по самой карточке), его останавливают методом event.stopPropagation().

→ Переходите к практике: Задание № 5 (папка «Задание № 5»).

2.6 Создание и удаление элементов

Новый DOM-узел создаётся методом document.createElement("тег") — на этом этапе элемент существует только в памяти и не виден на странице, пока не будет вставлен в дерево DOM одним из методов вставки:

```
const p = document.createElement("p");
p.textContent = "Новый абзац";

container.appendChild(p);   // вставить последним ребёнком
container.prepend(p);       // вставить первым ребёнком
```

Удаление узла из DOM выполняется его собственным методом remove() — современный способ, не требующий обращения к родителю (устаревший вариант: parent.removeChild(child)).

```
li.remove();
```

При работе со списками, элементы которых можно удалять, встаёт выбор между прямым обработчиком на каждый новый элемент и делегированием (2.5). Прямой обработчик проще для понимания, но его нужно вешать заново при создании каждого нового элемента; делегирование требует чуть более внимательного кода (closest, проверка на null), зато обработчик пишется один раз и работает для всех элементов, включая ещё не созданные.

→ Переходите к практике: Задание № 6 (папка «Задание № 6»).

2.7 Работа с формами

Разные типы полей формы читаются разными свойствами, и это частый источник ошибок:

— текстовое поле (<input type="text">, email, password, textarea) — значение читается через input.value, всегда строка;

— чекбокс (<input type="checkbox">) — состояние читается через input.checked (true/false), а НЕ через .value (value у чекбокса — фиксированная строка-атрибут, не меняющаяся при клике);

— radio-группа (несколько <input type="radio"> с одинаковым name) — выбранный вариант ищут селектором :checked: document.querySelector('input[name="plan"]:checked');

— выпадающий список (<select>) — выбранное значение читается через select.value, как у текстового поля.

Проверка формы перед отправкой обычно выполняется в обработчике события "submit" на самой форме, а не события "click" на кнопке — это гарантирует срабатывание и при нажатии Enter в поле. Типичный паттерн проверки — ранний выход (early return) из обработчика с выводом сообщения об ошибке для каждого условия по очереди:

```
      form.addEventListener("submit", function (event) {
        event.preventDefault();     // отменяем реальную отправку/перезагрузку
страницы

        if (nameInput.value.trim() === "") {
          errorBox.textContent = "Введите имя";
          return;                   // дальше не проверяем, пока не исправят
        }

        if (!agreeCheckbox.checked) {
          errorBox.textContent = "Нужно согласие";
          return;
        }

        errorBox.textContent = "";
        console.log("Форма валидна, можно отправлять");
      });
```

→ Переходите к практике: Задание № 7 (папка «Задание № 7»).

2.8 Таймеры (setTimeout, setInterval)

JavaScript умеет откладывать выполнение кода во времени — это часто нужно для всплывающих уведомлений, обратного отсчёта, автообновления данных. Для этого служат две функции, которые легко перепутать:

— setTimeout(fn, ms) — выполняет функцию fn ОДИН РАЗ, спустя ms миллисекунд, и на этом останавливается;

— setInterval(fn, ms) — выполняет функцию fn ПОВТОРНО, каждые ms миллисекунд, БЕСКОНЕЧНО, пока не будет остановлен вручную.

Обе функции возвращают числовой идентификатор запущенного таймера — его обязательно нужно сохранить в переменную, если таймер (особенно setInterval) предполагается когда-либо остановить методом clearInterval (для setTimeout — clearTimeout):

```
let timerId = null;

startBtn.addEventListener("click", function () {
  if (timerId !== null) return;     // защита от повторного запуска —
                                     // без неё каждый клик плодит новый
                                     // интервал, и счётчик начинает
                                     // ускоряться на глазах
  timerId = setInterval(function () {
    seconds--;
    display.textContent = seconds;
  }, 1000);
});

stopBtn.addEventListener("click", function () {
  clearInterval(timerId);
  timerId = null;
});
```

Показанная выше проверка timerId !== null — типичное и важное защитное условие: без неё повторный клик по кнопке запуска создаёт ВТОРОЙ, независимый интервал поверх первого, и оба продолжают работать параллельно, из-за чего счётчик начинает уменьшаться вдвое быстрее с каждым повторным запуском.

→ Переходите к практике: Задание № 8 (папка «Задание № 8»).

2.9 Fetch API

fetch() — встроенная функция браузера для отправки HTTP-запросов к серверу (например, за данными в формате JSON) без перезагрузки страницы. fetch() всегда возвращает Promise — объект, представляющий результат, который станет известен НЕ СРАЗУ, а через какое-то время (пока идёт сетевой запрос).

Работа с Promise-результатом fetch() возможна двумя равнозначными способами. Первый — цепочка .then():

```
fetch("https://jsonplaceholder.typicode.com/users/1")
  .then(function (response) {
    return response.json();      // тоже возвращает Promise —
распаковываем тело ответа как JSON
  })
  .then(function (data) {
    console.log(data.name);
  })
  .catch(function (error) {
    console.log("Ошибка сети:", error);
  });
```

Второй способ — async/await, который читается как обычный последовательный код и требует обёртки try/catch для обработки ошибок:

```
      async function loadUser() {
        try {
          const response = await
fetch("https://jsonplaceholder.typicode.com/users/1");
          const data = await response.json();
          console.log(data.name);
        } catch (error) {
          console.log("Ошибка сети:", error);
        }
      }
```

Важная особенность fetch(), часто становящаяся источником ошибок: блок catch (или reject самого Promise) срабатывает ТОЛЬКО при сбое СЕТИ (нет соединения, сервер недоступен и т. п.). Если сервер ответил, но с кодом ошибки (404 — не найдено, 500 — ошибка сервера и т. д.), fetch() считает запрос УСПЕШНЫМ и в catch не попадает — код HTTP-ответа нужно проверять вручную через свойство response.ok (true для кодов 200-299):

```
      const response = await fetch(url);
      if (!response.ok) {
        throw new Error("Ошибка сервера: " + response.status);
      }
      const data = await response.json();
```

→ Переходите к практике: Задание № 9 (папка «Задание № 9»).

2.10 JSON

JSON (JavaScript Object Notation) — текстовый формат представления структурированных данных, используемый как основной формат обмена данными между браузером и сервером (в том числе в ответах fetch(), см. 2.9), а также для хранения данных в localStorage.

Преобразование между JS-объектом и JSON-строкой выполняют две обратные друг другу функции:

```
      const person = { name: "Аня", age: 20 };

      const json = JSON.stringify(person);   // объект -> строка:
'{"name":"Аня","age":20}'
      const back = JSON.parse(json);         // строка -> обратно объект: {
name: "Аня", age: 20 }
```

localStorage — встроенное в браузер постоянное хранилище ключ-значение, сохраняющее данные между перезагрузками страницы и закрытиями браузера. Ключевое ограничение localStorage — он умеет хранить ТОЛЬКО СТРОКИ. Попытка сохранить обычный объект напрямую (без JSON.stringify) не вызовет ошибку, но приведёт к молчаливой потере данных — объект будет преобразован в бесполезную строку "[object Object]":

```
      // НЕПРАВИЛЬНО — потеря данных:
      localStorage.setItem("user", person);        // сохранится строка
"[object Object]"

      // ПРАВИЛЬНО:
      localStorage.setItem("user", JSON.stringify(person));
      const saved = JSON.parse(localStorage.getItem("user"));
```

→ Переходите к практике: Задание № 10 (папка «Задание № 10»).

3 Порядок выполнения лабораторной работы

Лабораторная работа состоит из десяти практических заданий (по темам подразделов 2.1-2.10) и итогового задания (раздел 5). Каждое задание представлено файлами varXX.html и varXX.js, где XX — номер варианта от 01 до 30; вся логика находится в файле .js, а .html содержит только готовую разметку и подключает соответствующий скрипт.

Каждый файл varXX.js содержит блоки TODO, расположенные по возрастанию сложности:

— лёгкий уровень;

— средний уровень;

— сложный уровень;

— отдельные задания (там, где это уместно теме) — задача уровня «найди и исправь ошибку» в уже написанном коде.

Над каждым TODO дано текстовое описание того, что должен реализовать обучающийся, а также — для лёгкого и среднего уровня — развёрнутые подсказки с указанием конкретных методов и свойств, которые потребуются. Задания сложного уровня и итоговое задание подсказок не содержат.

Проверка результата выполняется открытием HTML-файла варианта через расширение Live Server и наблюдением за поведением страницы, а также проверкой сообщений в консоли браузера (DevTools → Console) на предмет ошибок.

4 Самостоятельная работа

Номер варианта уточняется у преподавателя. Самостоятельная работа считается выполненной при сдаче всех десяти заданий и итогового задания своего варианта. Соответствие заданий, папок и файлов приведено в таблице 2.

Таблица 2 — Соответствие заданий, папок и имён файлов

№	Тема задания	Папка	Файл(ы) варианта
1	DOM и выбор элементов	Задание №1	varXX.html, varXX.js
2	Изменение содержимого и атрибутов	Задание №2	varXX.html, varXX.js
3	Стили через JS	Задание №3	varXX.html, varXX.js
4	События (addEventListener)	Задание №4	varXX.html, varXX.js
5	Всплытие и делегирование событий	Задание №5	varXX.html, varXX.js
6	Создание и удаление элементов	Задание №6	varXX.html, varXX.js
7	Работа с формами	Задание №7	varXX.html, varXX.js
8	Таймеры (setTimeout, setInterval)	Задание №8	varXX.html, varXX.js
9	Fetch API	Задание №9	varXX.html, varXX.js
10	JSON (JSON.stringify / JSON.parse)	Задание №10	varXX.html, varXX.js

Здесь и далее XX — номер варианта от 01 до 30. Все папки заданий, а также папка «Итоговое задание», находятся внутри папки «Лабораторная работа №3».

Результаты выполнения всех заданий и итогового задания оформляются обучающимся в виде отчёта в соответствии с требованиями ГОСТ ГУАП и сдаются вместе с исходными файлами.

Критерии оценки самостоятельной работы:

— корректность работы скрипта по каждому из десяти заданий и итоговому заданию своего варианта;

— отсутствие необработанных ошибок в консоли браузера;

— осмысленное использование делегирования событий там, где это уместно (темы 2.5-2.6);

— корректная обработка ошибок сети/сервера в заданиях по Fetch API (проверка response.ok);

— соблюдение принятых на курсе соглашений об именовании переменных и функций;

— оформление отчёта в соответствии с требованиями ГОСТ ГУАП;

— защита лабораторной работы (устный ответ на вопросы преподавателя по теоретической части).

5 Итоговое задание

Итоговое задание лабораторной работы № 3 логически продолжает итоговое задание лабораторной работы № 2: обучающийся берёт СОБСТВЕННЫЙ, уже свёрстанный им сайт своего тематического варианта (созданный в разделе «Итоговое задание» лабораторной работы № 2) и оживляет его при помощи JavaScript.

В отличие от заданий №1-10, итоговое задание не содержит собственной HTML/CSS-разметки — оно состоит только из одного JavaScript-файла на вариант (varXX.js в папке «Итоговое задание»). Файл нужно скопировать целиком в СВОЙ проект лабораторной работы № 2 и подключить перед закрывающим тегом </body> собственной страницы: <script src="varXX.js"></script>. Работать следует именно со своей структурой и своим CSS-файлом лабораторной работы № 2, а не создавать новую разметку — сам файл варианта содержит в комментариях напоминание об исходной структуре страницы (навигация, карточки услуг, промо-блок, форма) и явно указывает, какие новые элементы (например, кнопку «в избранное», блок с отзывом, счётчик обратного отсчёта) нужно самостоятельно добавить в свою разметку.

Требования к итоговому заданию опираются на приёмы, изученные во всех десяти заданиях лабораторной работы № 3, и типично включают:

— делегирование событий (2.5) для навигации и карточек — переключение классов active/selected при клике;

— динамическое создание и удаление элементов (2.6) — например, список избранного, куда элементы добавляются и из которого удаляются по клику;

— работу с формой (2.7) — проверку полей при отправке и вывод статуса без перезагрузки страницы;

— таймер (2.8) — например, обратный отсчёт до конца акции в промо-блоке;

— запрос к серверу через fetch (2.9) — подгрузку дополнительных данных (например, отзыва) по кнопке, с обработкой ошибки сети;

— сохранение состояния в localStorage через JSON (2.10) — например, чтобы список избранного не терялся при перезагрузке страницы.

Подсказок по коду файл варианта не содержит — это итоговая, самостоятельная работа, опирающаяся на материал заданий №1-10. Итоговое задание входит в состав отчёта по самостоятельной работе (раздел 4) и защищается вместе с ним.

ЗАКЛЮЧЕНИЕ

Выполнение настоящей лабораторной работы обеспечивает освоение базовых приёмов клиентского JavaScript: выбора и изменения элементов DOM, обработки событий и делегирования, динамического создания и удаления элементов, работы с формами, таймеров, а также обмена данными с сервером через Fetch API и формат JSON. Итоговое задание, объединяющее эти приёмы на собственном сайте обучающегося из лабораторной работы № 2, закладывает основу для перехода к изучению React — там тот же материал (состояние, события, рендер по данным) выражается через декларативный, а не императивный подход к изменению интерфейса.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. MDN Web Docs: Введение в DOM [Электронный ресурс]. — Режим доступа: https://developer.mozilla.org/ru/docs/Web/API/Document_Object_Model/Introduction

2. MDN Web Docs: EventTarget.addEventListener() [Электронный ресурс]. — Режим доступа: https://developer.mozilla.org/ru/docs/Web/API/EventTarget/addEventListener

3. MDN Web Docs: Всплытие и перехват событий [Электронный ресурс]. — Режим доступа: https://developer.mozilla.org/ru/docs/Learn_web_development/Core/Scripting/Event_bubbling

4. MDN Web Docs: Using the Fetch API [Электронный ресурс]. — Режим доступа: https://developer.mozilla.org/ru/docs/Web/API/Fetch_API/Using_Fetch

5. MDN Web Docs: JSON [Электронный ресурс]. — Режим доступа: https://developer.mozilla.org/ru/docs/Web/JavaScript/Reference/Global_Objects/JSON

6. MDN Web Docs: Window.localStorage [Электронный ресурс]. — Режим доступа: https://developer.mozilla.org/ru/docs/Web/API/Window/localStorage

7. javascript.info: События изменения элементов управления формы [Электронный ресурс]. — Режим доступа: https://learn.javascript.ru/events-change-input

8. ГОСТ Р 7.0.100-2018. Библиографическая запись. Библиографическое описание. — Введ. 2019-07-01. — М.: Стандартинформ, 2018.